



LIBYAN INTERNATIONAL MEDICAL UNIVERSITY (LIMU)

SOFTWARE ENGINEERING

CAPSTONE II

Milestone III (design):

Automated Financial Transaction Reconciliation System (AFTRS)

Amin Osman

Student ID no.: 4109

Spring Semester 2025/2026

Date of Submission: 07-05-2026

1. Overview

This chapter details the system design phase for the Automated Financial Transaction Reconciliation System (AFTRS). While the previous milestone (Software Requirements Specification) defined *what* the system must do, this chapter defines exactly *how* the system will do it. It provides the architectural blueprint required in the development phase to construct the system using C#, ASP.NET Core MVC, and Microsoft SQL Server. This chapter includes comprehensive visual models of the system's dynamic behaviours, static structures, and database schematics, ensuring that all complex algorithms—specifically the deterministic matching and fuzzy logic engines—are mapped out clearly before implementation begins.

2. Introduction about the Design Phase

In the software engineering life cycle, particularly within the strict Waterfall methodology adopted for the AFTRS project, the Design Phase is the critical bridge between requirements gathering and actual coding.

The primary goal of this phase is to transform the validated functional and non-functional requirements into a technical framework. Because financial systems require absolute precision, auditability, and deterministic logic, the design phase eliminates ambiguity. It dictates the architecture of the application, the structure of the relational database, and the step-by-step flow of data between the user interface and the back-end servers.

The AFTRS design is divided into two main categories:

- 1.Process Design:** Mapping the logical flow of tasks, such as how a file is parsed, how a discrepancy is resolved, and how the audit log captures data.
- 2.Structural Design:** Defining the Entity Framework Core (EF Core) database models, the C# classes, and the physical deployment environment of the web server.

2.1 Unified Modeling Language (UML)

To document the system's architecture and processes, this project utilizes the Unified Modeling Language (UML). UML is the industry-standard visual modeling language used in object-oriented software engineering. It provides a standardized way to visualize the design of a system, making complex code structures and workflows understandable to both developers and stakeholders.

For the AFTRS, UML is deployed to represent different perspectives of the system:

- **Behavioural Diagrams (Activity and Sequence Diagrams):** These are used to model the dynamic behaviour of the system. They map out the timeline of operations, showing how the Financial Manager interacts with the system to upload files, trigger the reconciliation engine, and generate strategic reports.
- **Structural Diagrams (Class, Component, and Deployment Diagrams):** These illustrate the static architecture of the application. They define the C# objects (Classes), the relationships between database tables, the major software modules (Components), and how

the application is hosted on the server infrastructure (Deployment).

- **Interaction Diagrams (Collaboration Diagram):** These show how the Model, View, and Controller (MVC) layers communicate with each other during core system processes.

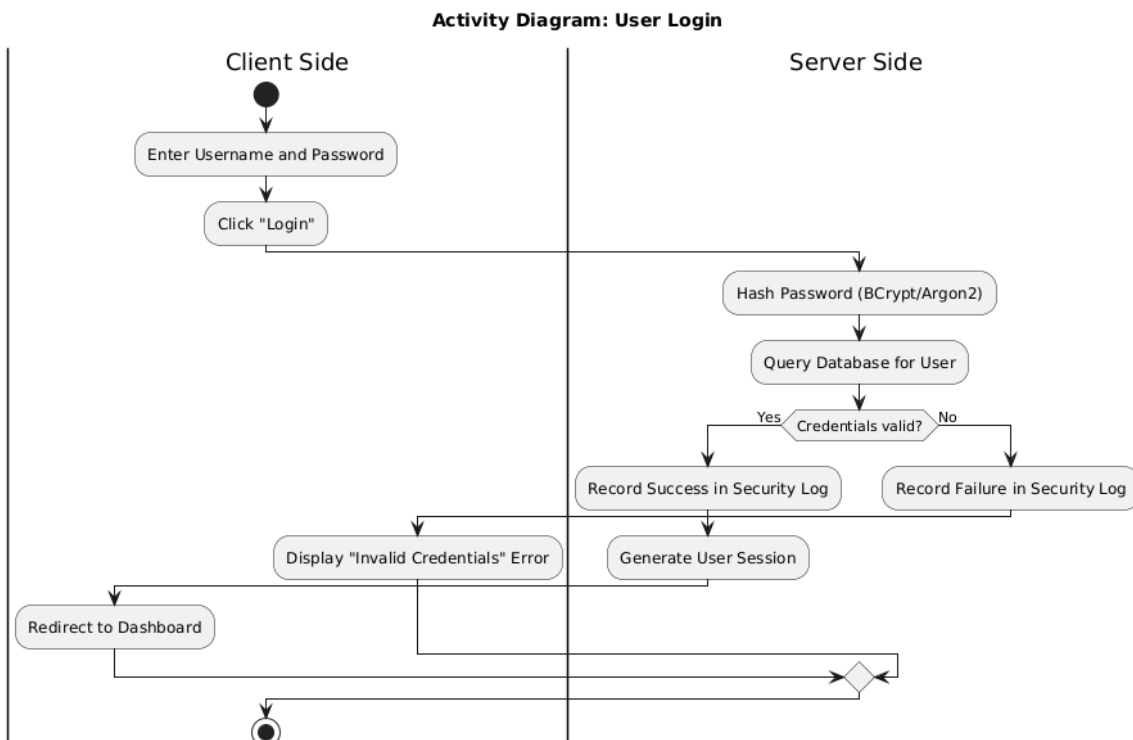
By utilizing these UML standards, the AFTRS establishes a rigid, verifiable, and highly maintainable blueprint, ensuring that the final coded product aligns perfectly with the compliance and efficiency goals of the organization.

3. Activity Diagrams

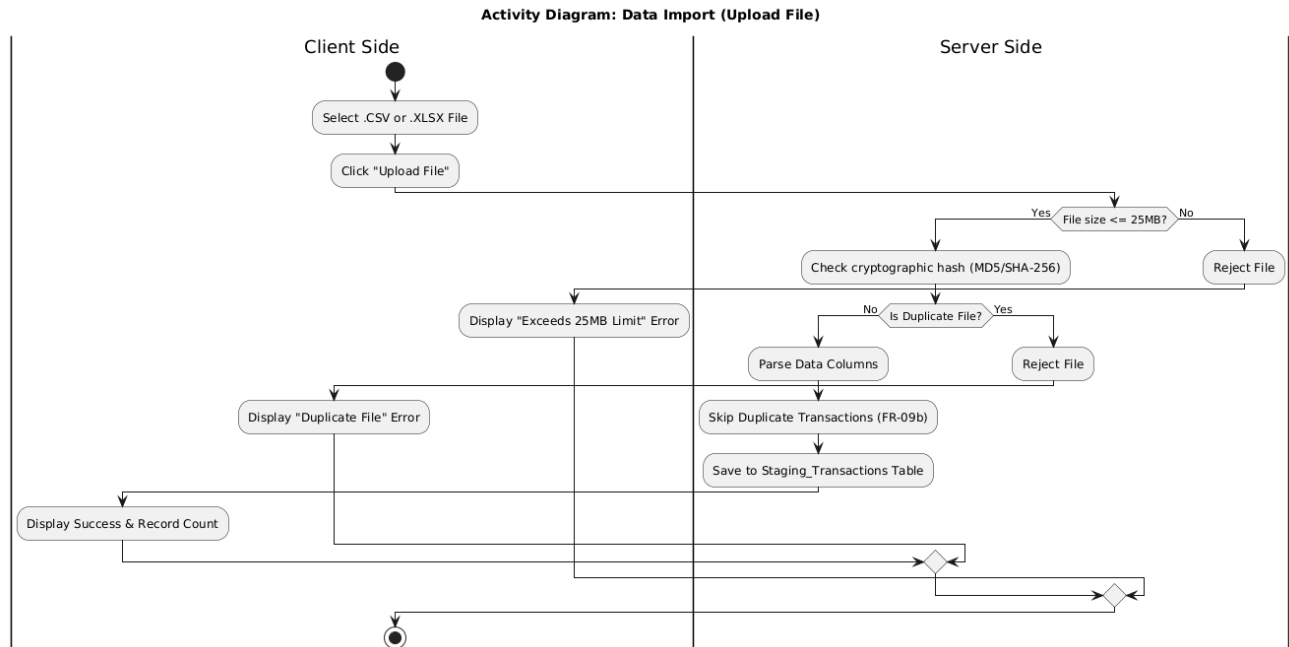
Activity diagrams are graphical representations of workflows of stepwise activities and actions with support for choice, iteration, and concurrency. In the AFTRS design phase, these diagrams map out the exact logical sequence of operations for each major system feature, detailing how the user interacts with the system and how the system processes data in the background.

Below are the activity diagrams modelling the core processes of the AFTRS.

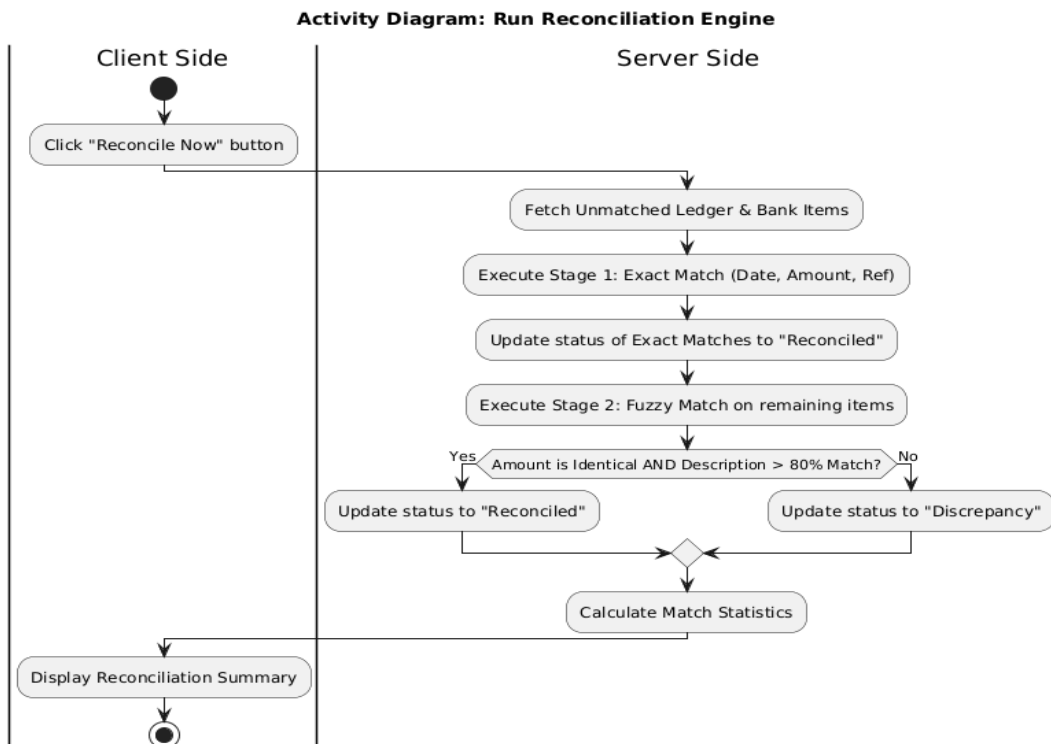
3.1 Activity Diagram: User Authentication (Login)



3.2 Data Import & Parsing (UC-03/04 & FR-08, FR-09a)

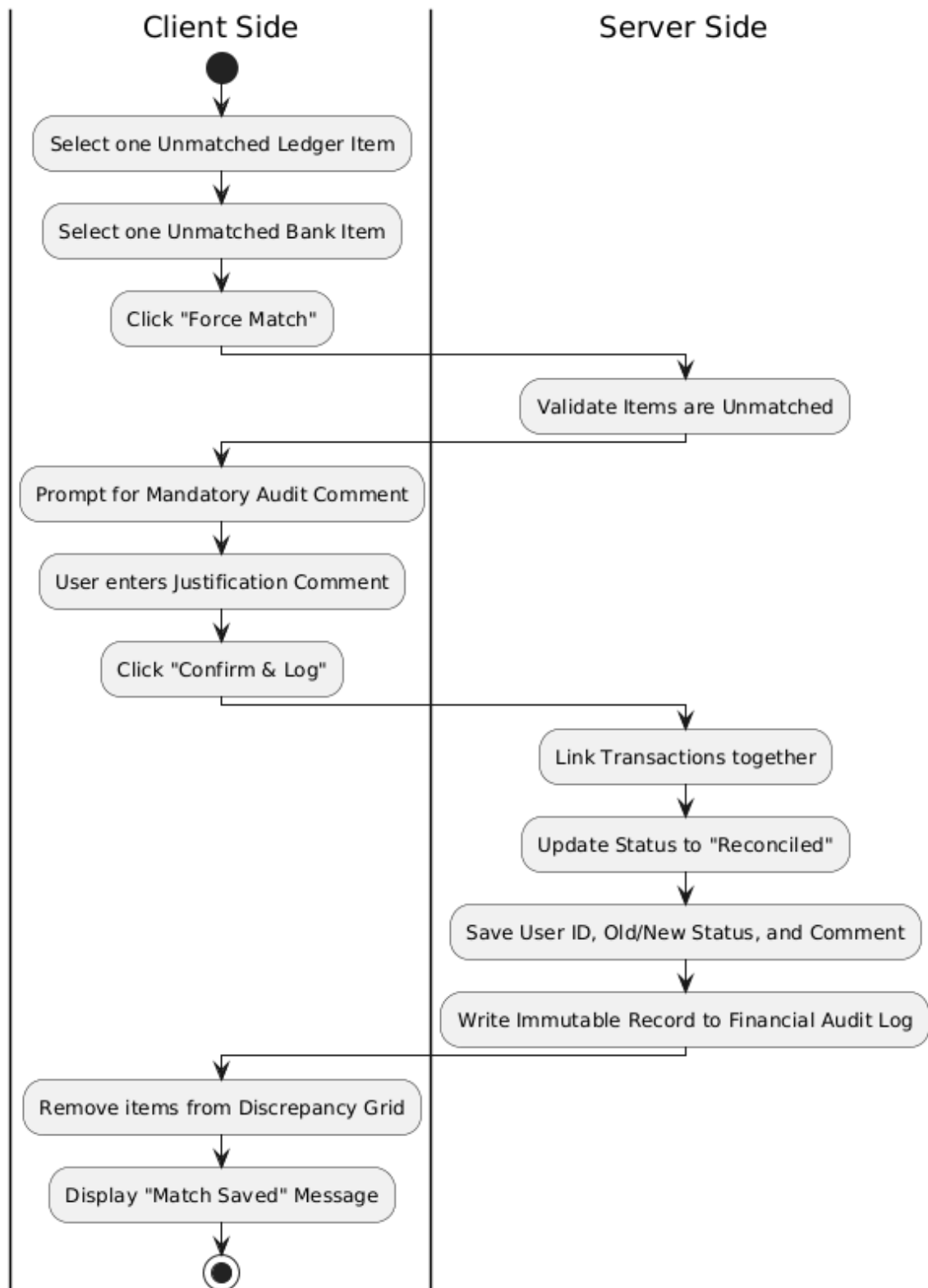


3.3 Run Reconciliation Engine (UC-05 & FR-10, FR-11)

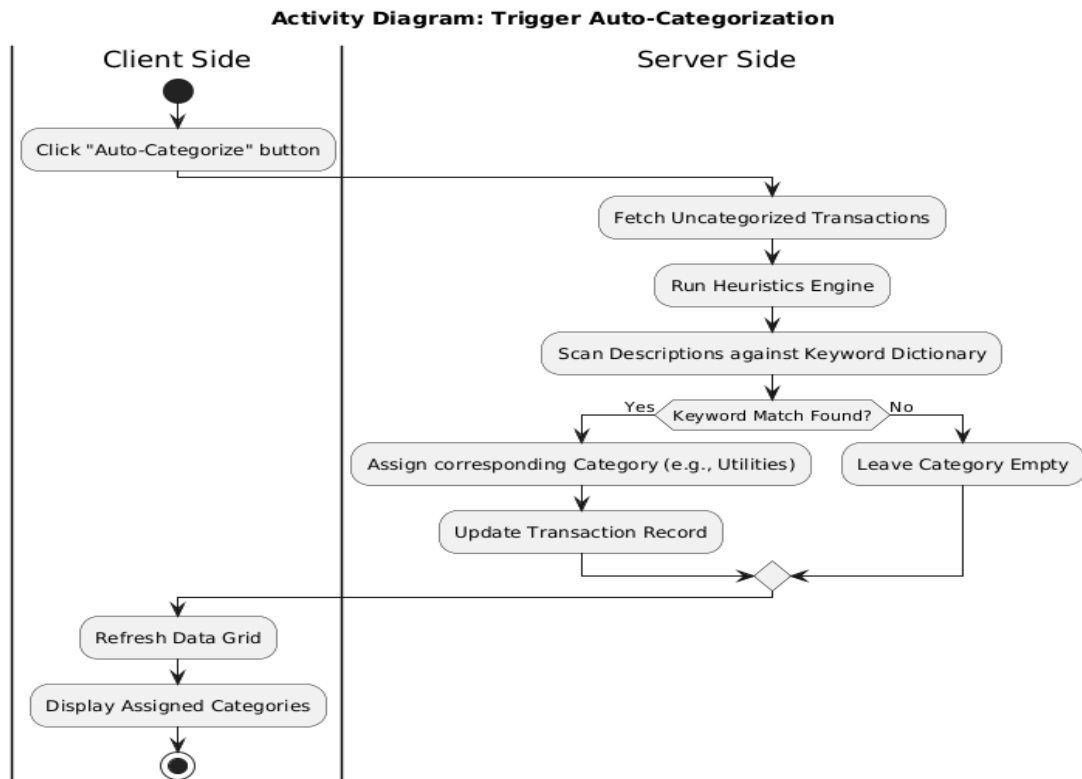


3.4 Manually Resolve Discrepancy (UC-07/08 & FR-16, FR-17)

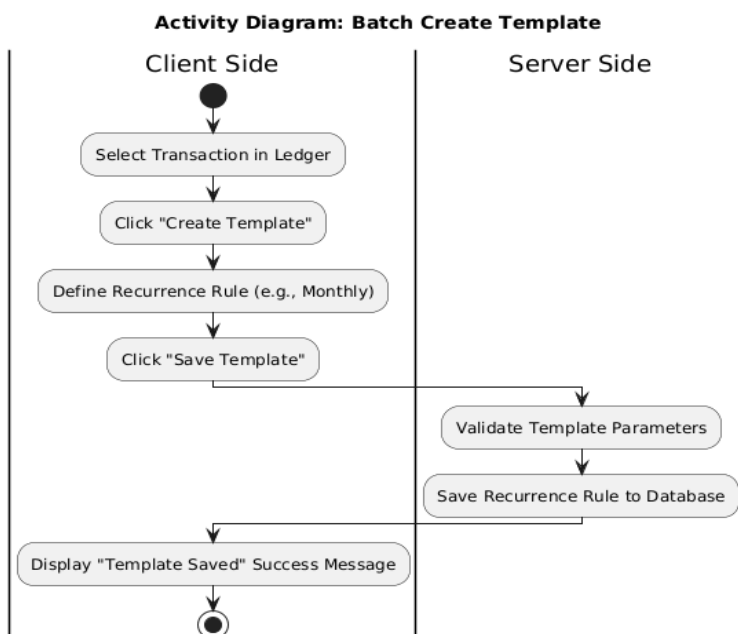
Activity Diagram: Manually Resolve Discrepancy



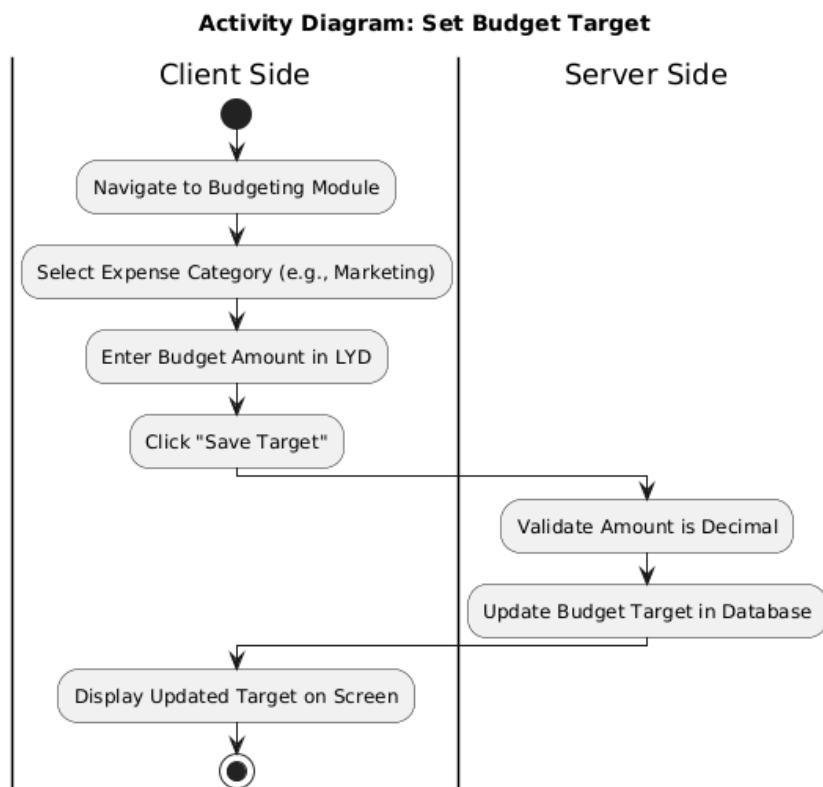
3.5 Trigger Auto-Categorization (UC-10 & FR-13)



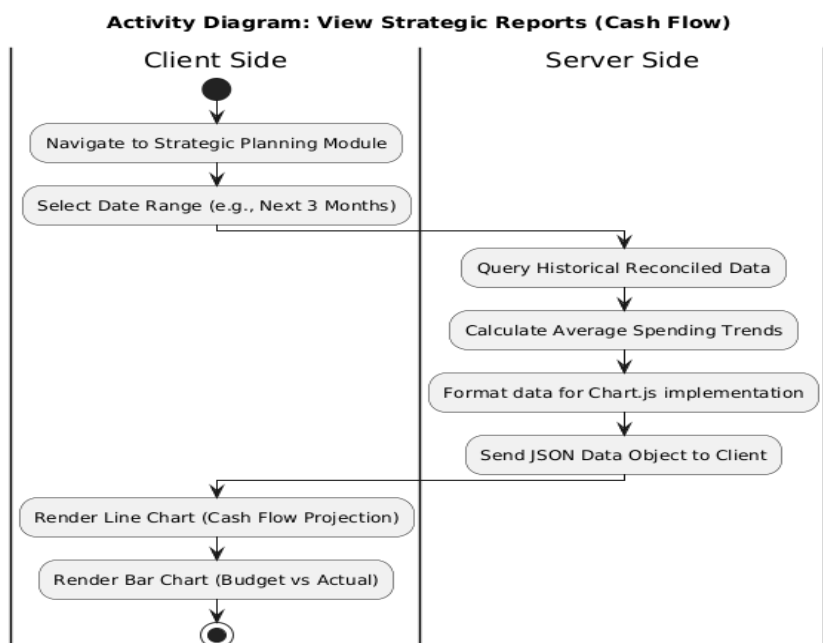
3.6 Batch Create Template (UC-09)



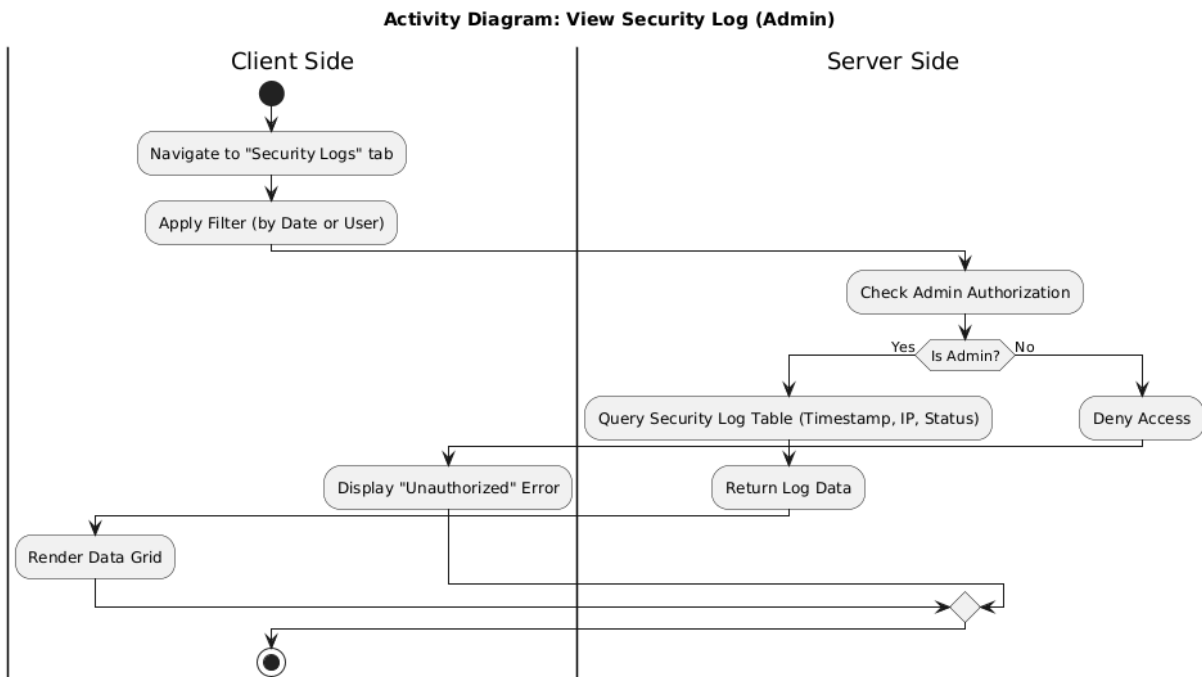
3.7 Set Budget Target (UC-11 & FR-18)



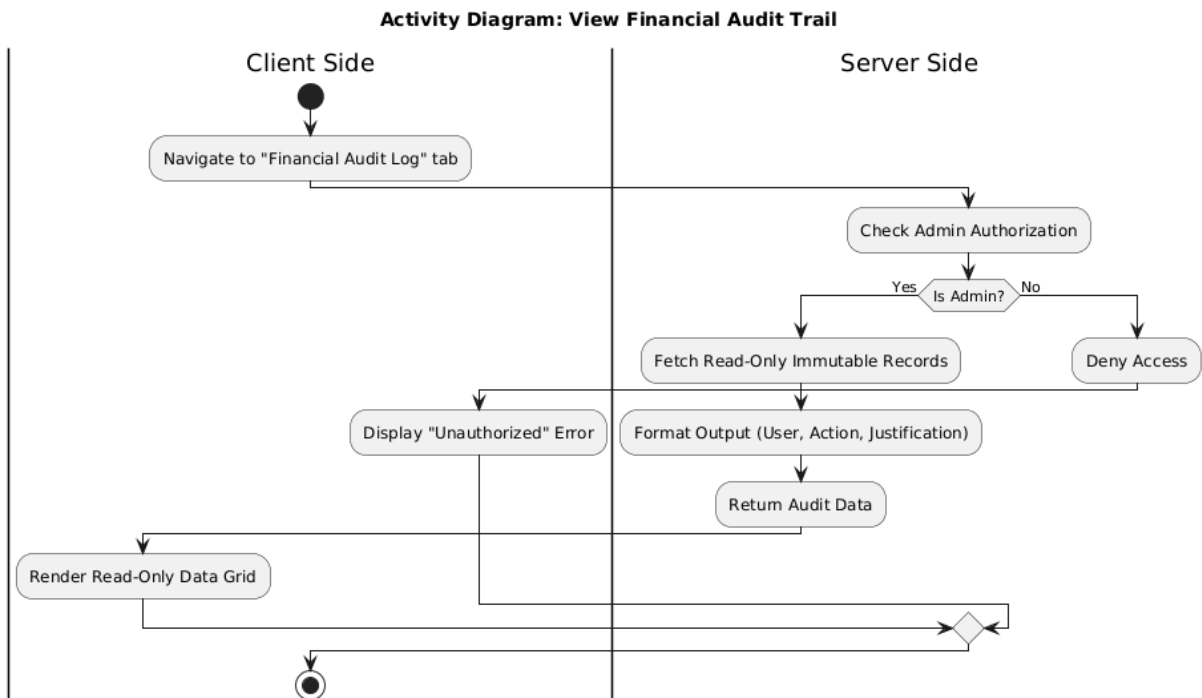
3.8 View Strategic Reports (UC-13 & FR-20)



3.9 View Security Log (Admin) (UC-15 & FR-04)



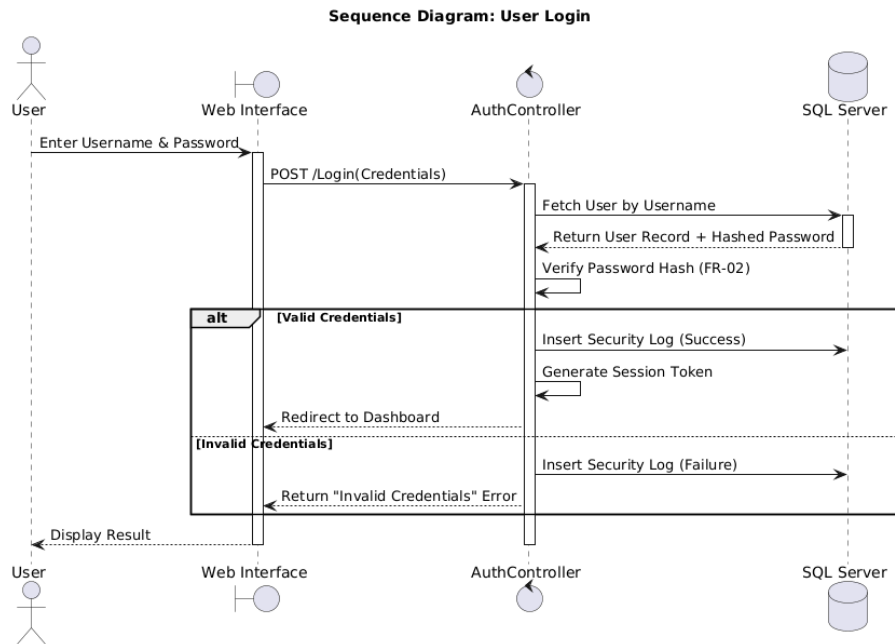
3.10 View Financial Audit Trail (Admin) (UC-16 & FR-17)



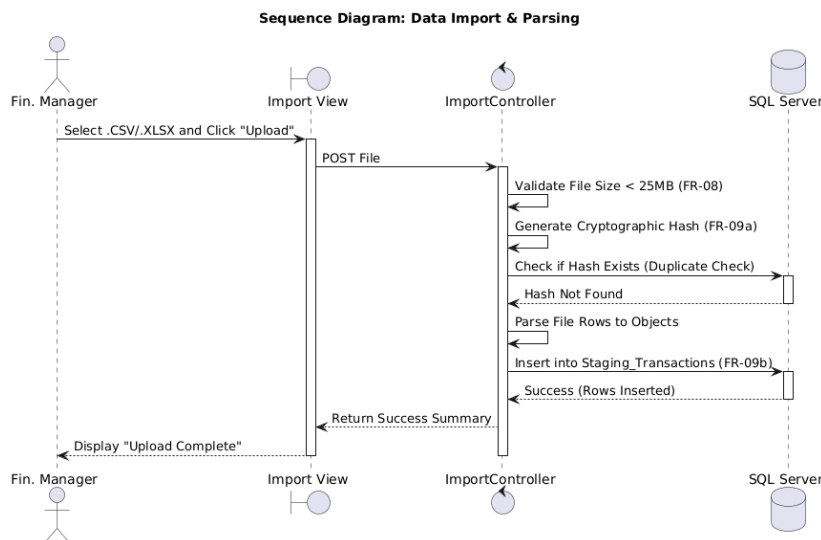
4. Sequence diagrams

In software engineering, while Activity Diagrams show the *flow of actions*, Sequence Diagrams show the *flow of messages over time* between the different components of your architecture.

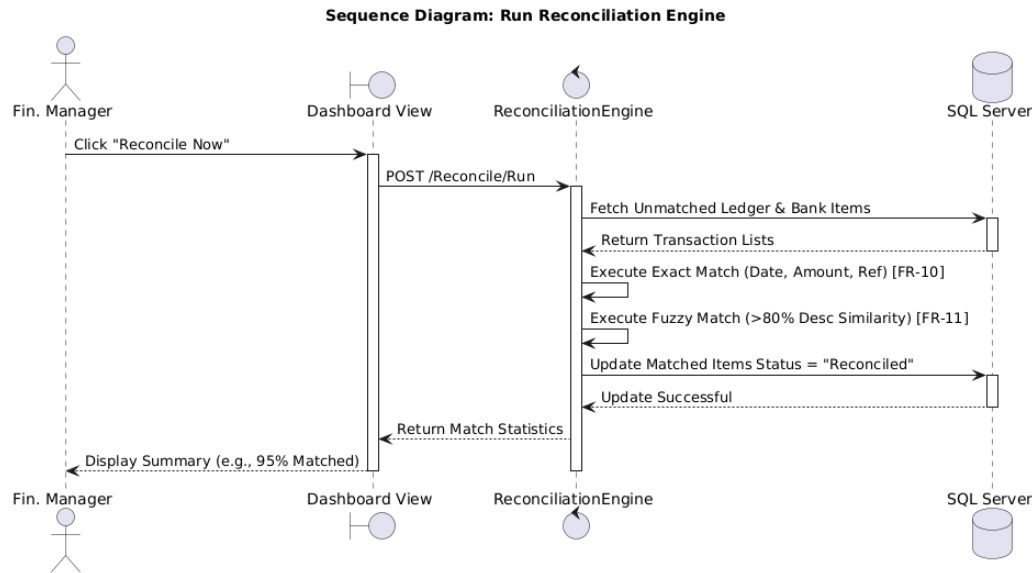
4.1 User Login (UC-01)



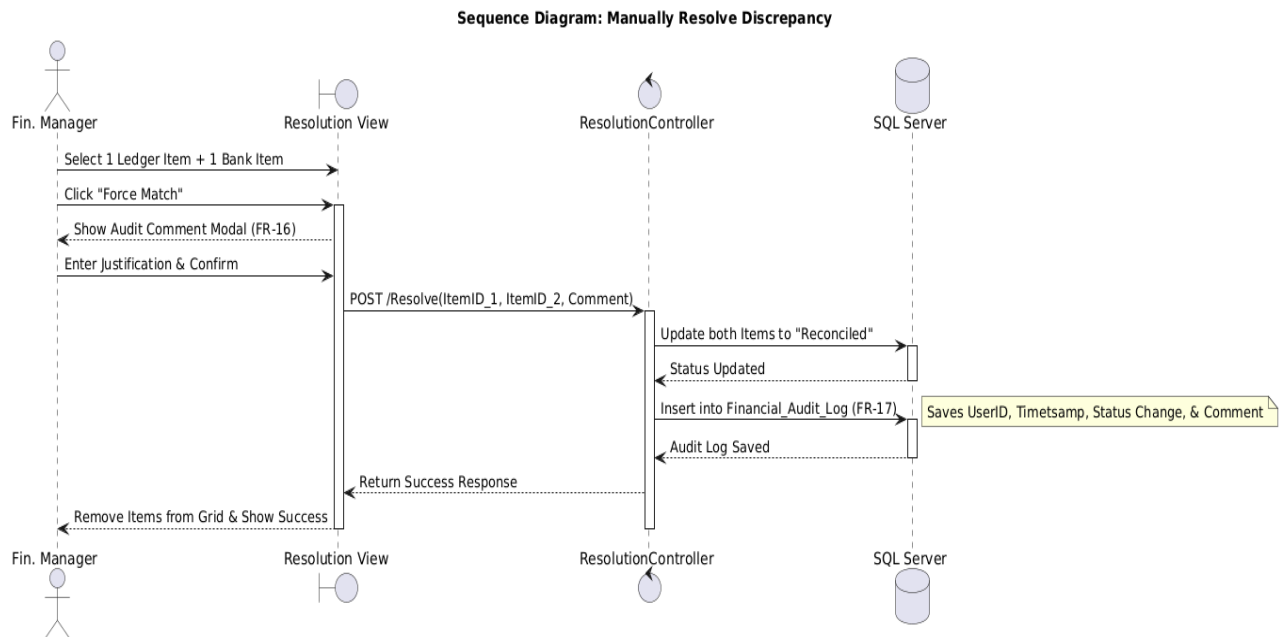
4.2 Upload Data Files (UC-03 & UC-04)



4.3 Run Reconciliation Engine (UC-05)

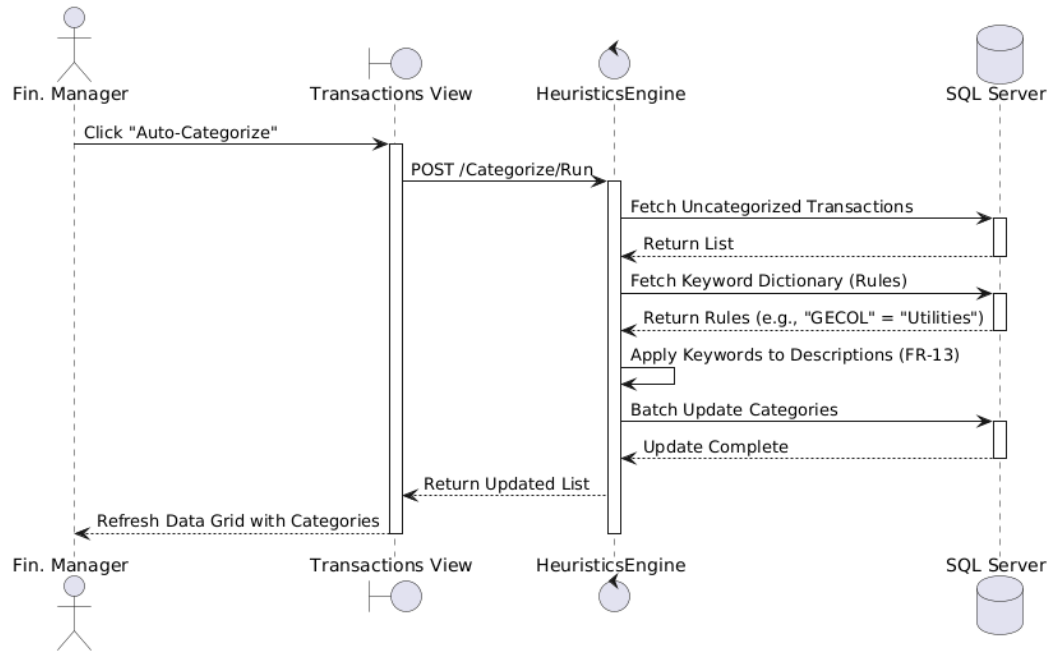


4.4 Manually Resolve Discrepancy (UC-07 & UC-08)



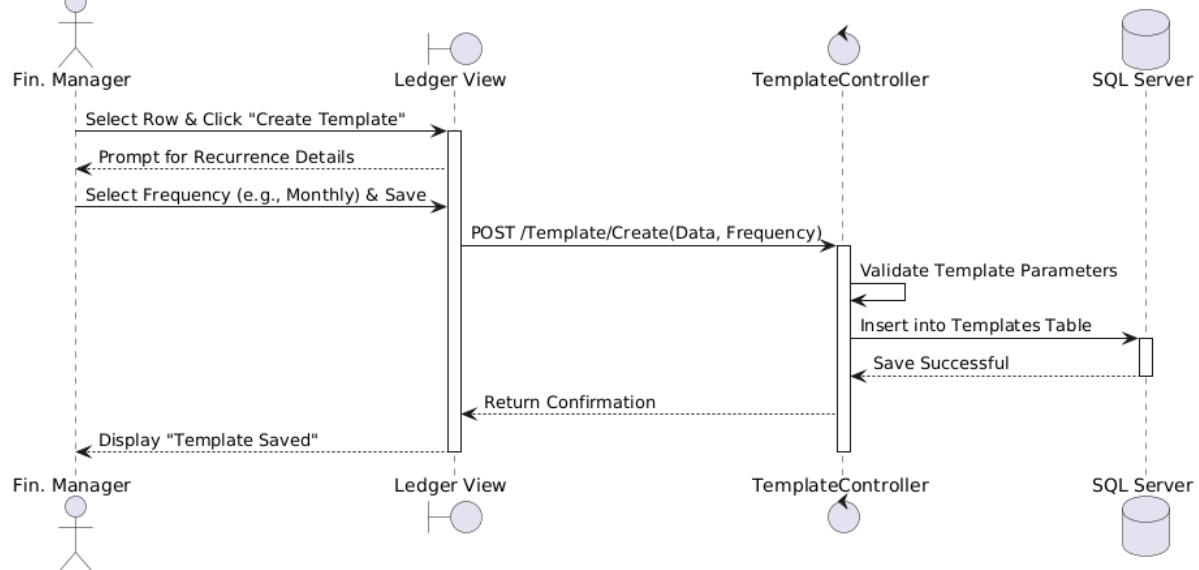
4.5 Trigger Auto-Categorization (UC-10)

Sequence Diagram: Trigger Auto-Categorization



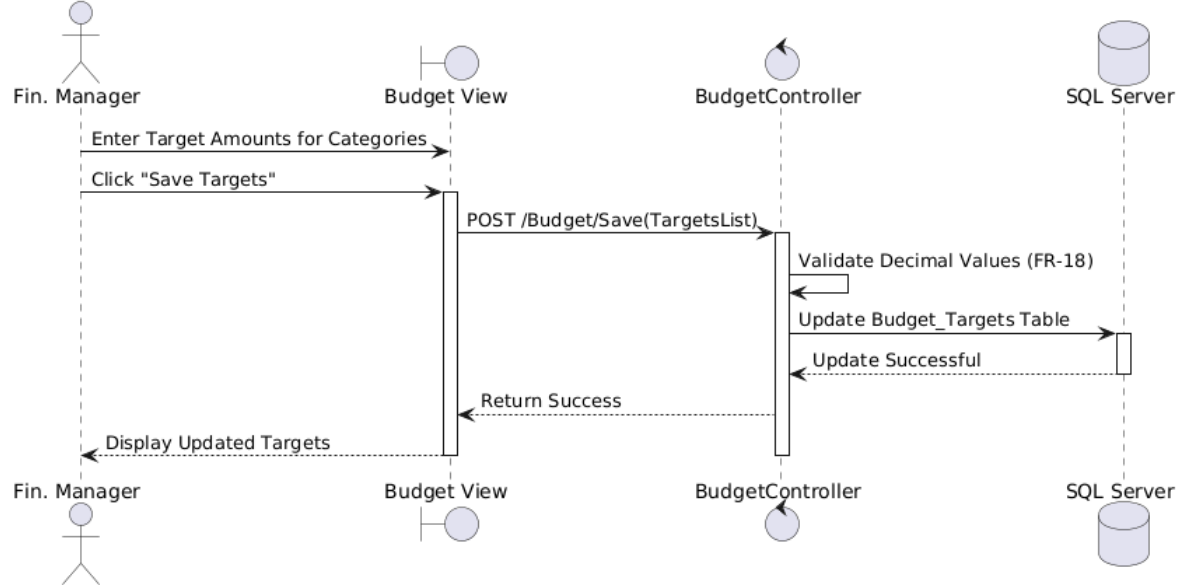
4.6 Batch Create Template (UC-09)

Sequence Diagram: Batch Create Template



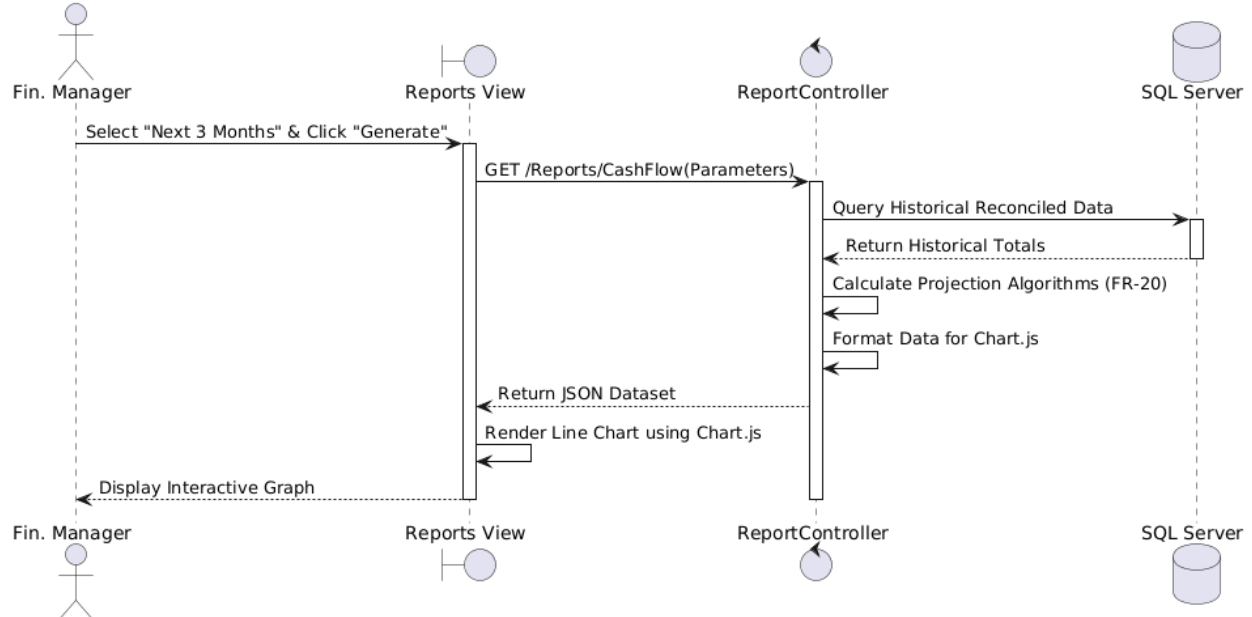
4.7 Set Budget Targets (UC-11)

Sequence Diagram: Set Budget Targets



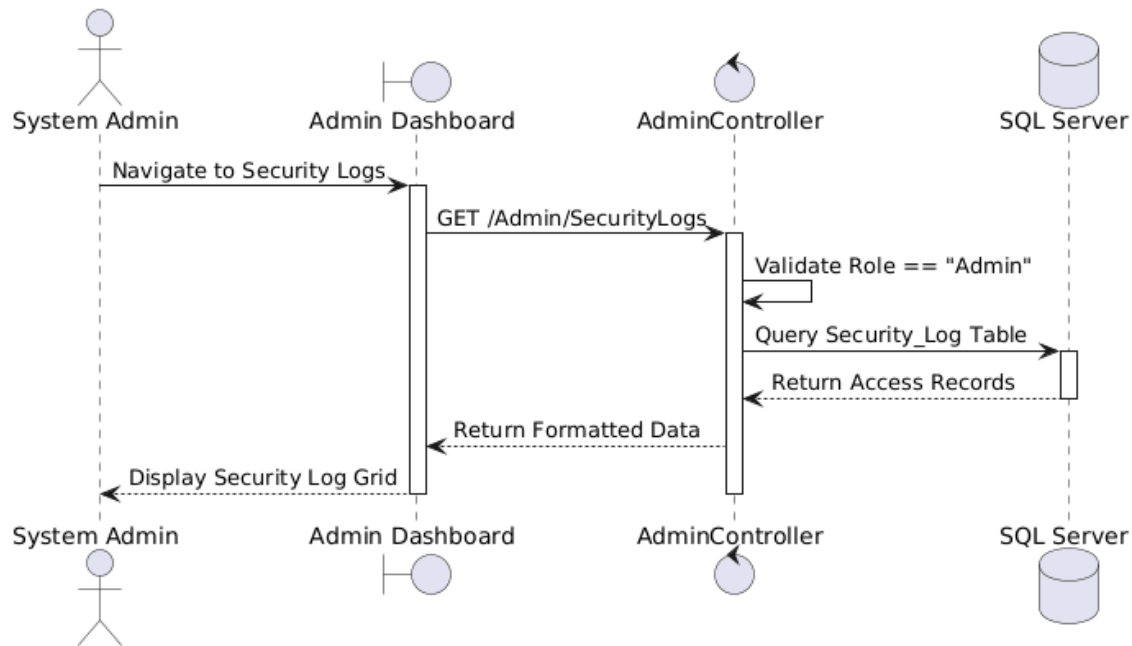
4.8 View Cash Flow Projections (UC-13)

Sequence Diagram: View Cash Flow Projections



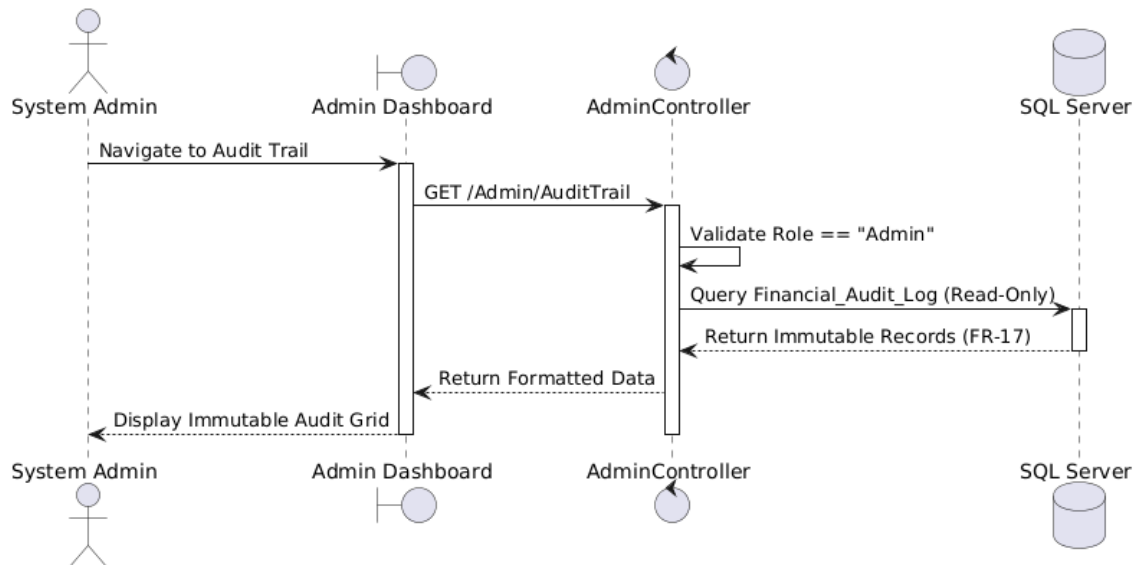
4.9 View Security Log (UC-15)

Sequence Diagram: View Security Log (Admin)



4.10 View Financial Audit Trail (UC-16)

Sequence Diagram: View Financial Audit Trail

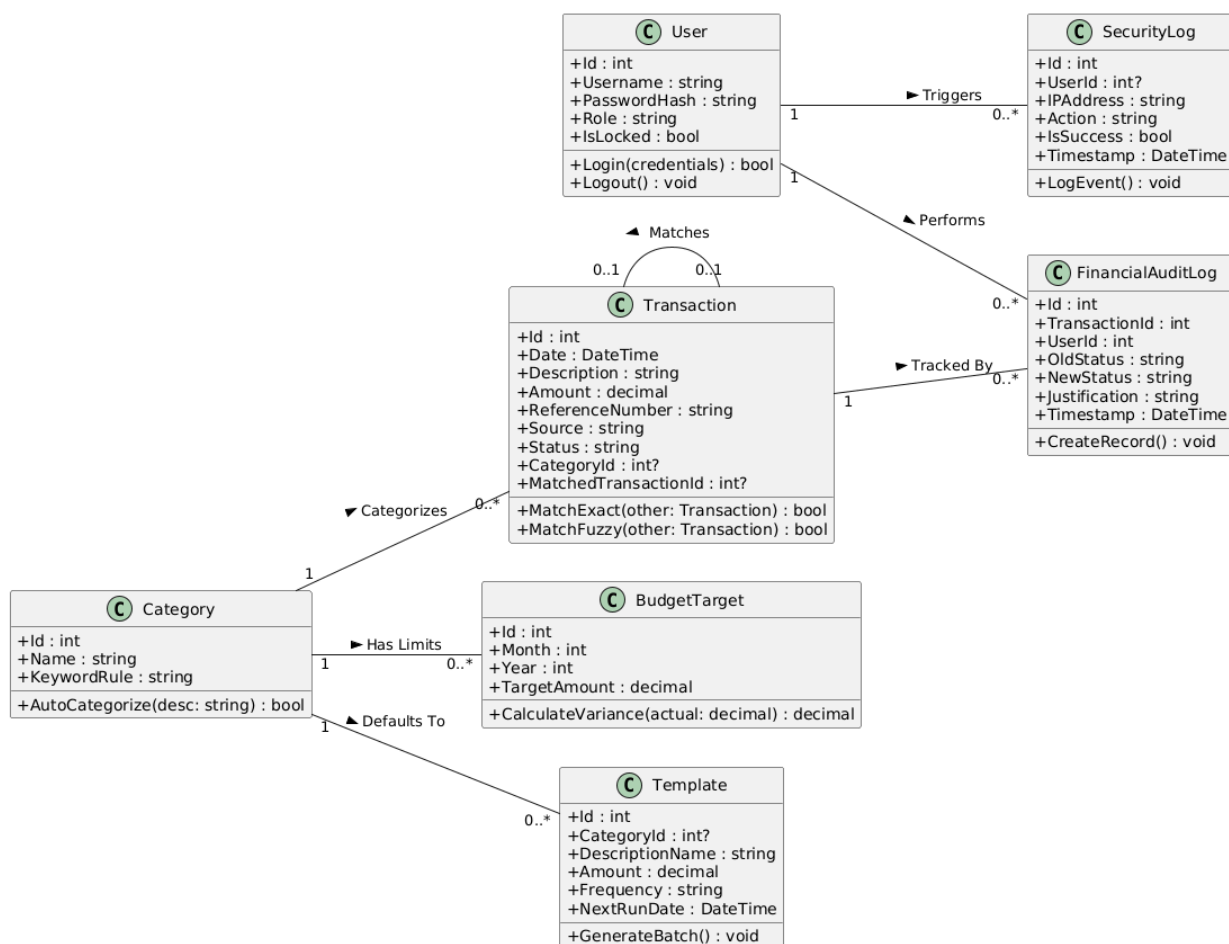


While the previous diagrams showed *actions* and *time*, these diagrams show the static architecture of your system. They define exactly what C# classes you will code and what SQL Server tables Entity Framework Core (EF Core) will generate.

5. Class Diagram

The Class Diagram provides an object-oriented structural blueprint of the AFTRS application. Since the system is built using ASP.NET Core, these classes represent the Models in the MVC architecture. The diagram highlights the attributes (data fields), methods (functions), and the multiplicity relationships between the core entities such as Users, Transactions, and the Compliance Logs. Notably, it demonstrates how the FinancialAuditLog strictly references the Transaction and User classes to enforce the legal Non-Repudiation requirement.

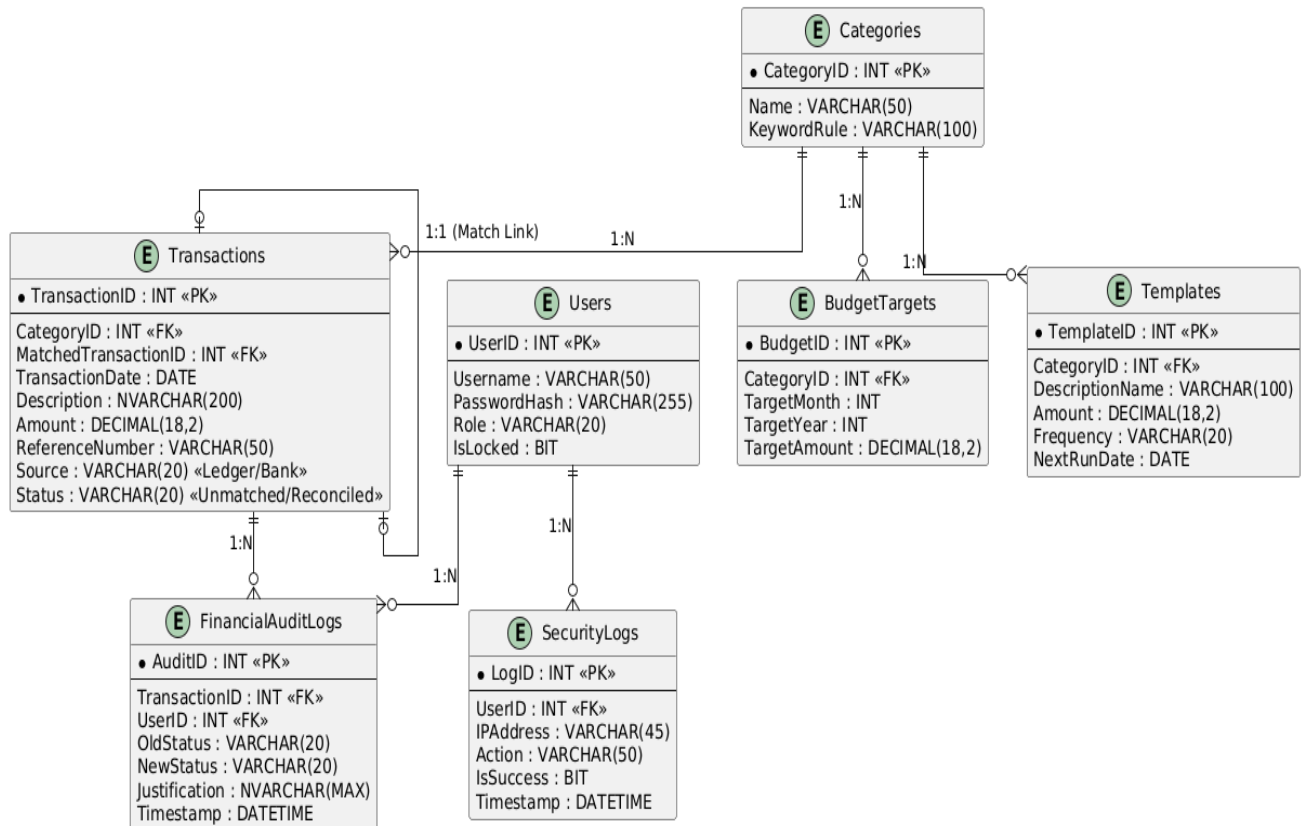
AFTRS Class Diagram (Domain Models)



6. Database Schema

The Database Schema represents the relational data model hosted on Microsoft SQL Server. Using Entity Framework Core (EF Core) as the Object-Relational Mapper (ORM), the C# classes from the Class Diagram are translated into SQL tables. The Entity-Relationship (ER) diagram outlines the Primary Keys (PK), Foreign Keys (FK), and data types required to maintain data integrity. Decimal (18,2) is explicitly used for all monetary fields to prevent floating-point rounding errors during reconciliation.

AFTRS Database Schema (Entity-Relationship)

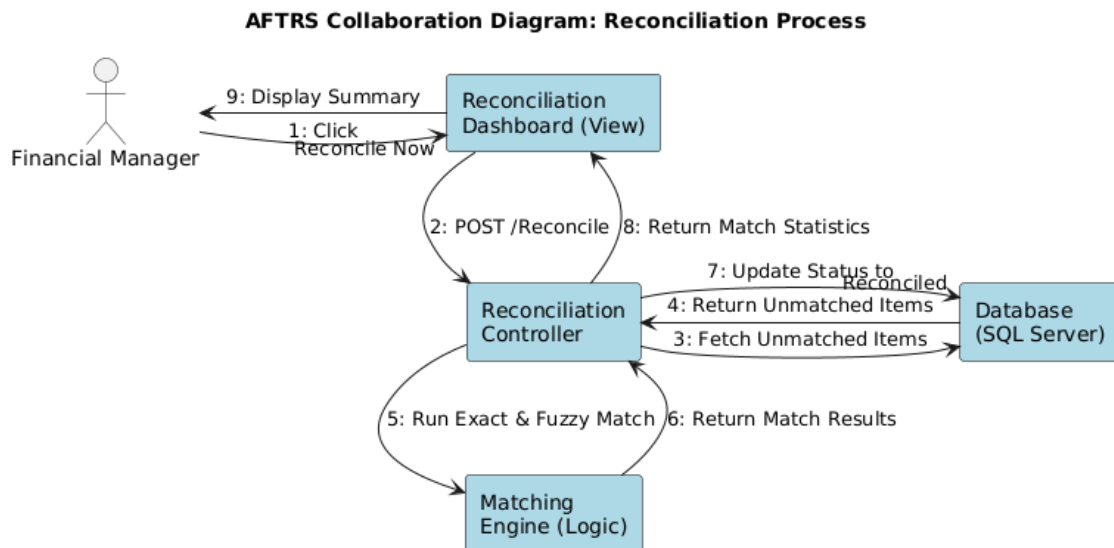


These diagrams take a step back from the specific code and show the "Big Picture." They explain how the major pieces of your software fit together, how they communicate, and where they will physically live on the university's hardware.

7. Collaboration Diagram

A Collaboration Diagram (also known as a Communication Diagram in UML 2.x) illustrates the interactions between objects or parts in terms of sequenced messages. While sequence diagrams focus on time, collaboration diagrams focus on the structural organization of the objects that send and receive messages.

For the AFTRS, this diagram models the core Reconciliation Process. It demonstrates how the Financial Manager interacts with the View, which triggers the Controller, invokes the complex Matching Engine logic, and finally updates the SQL Database, numbered in the exact sequence of execution.

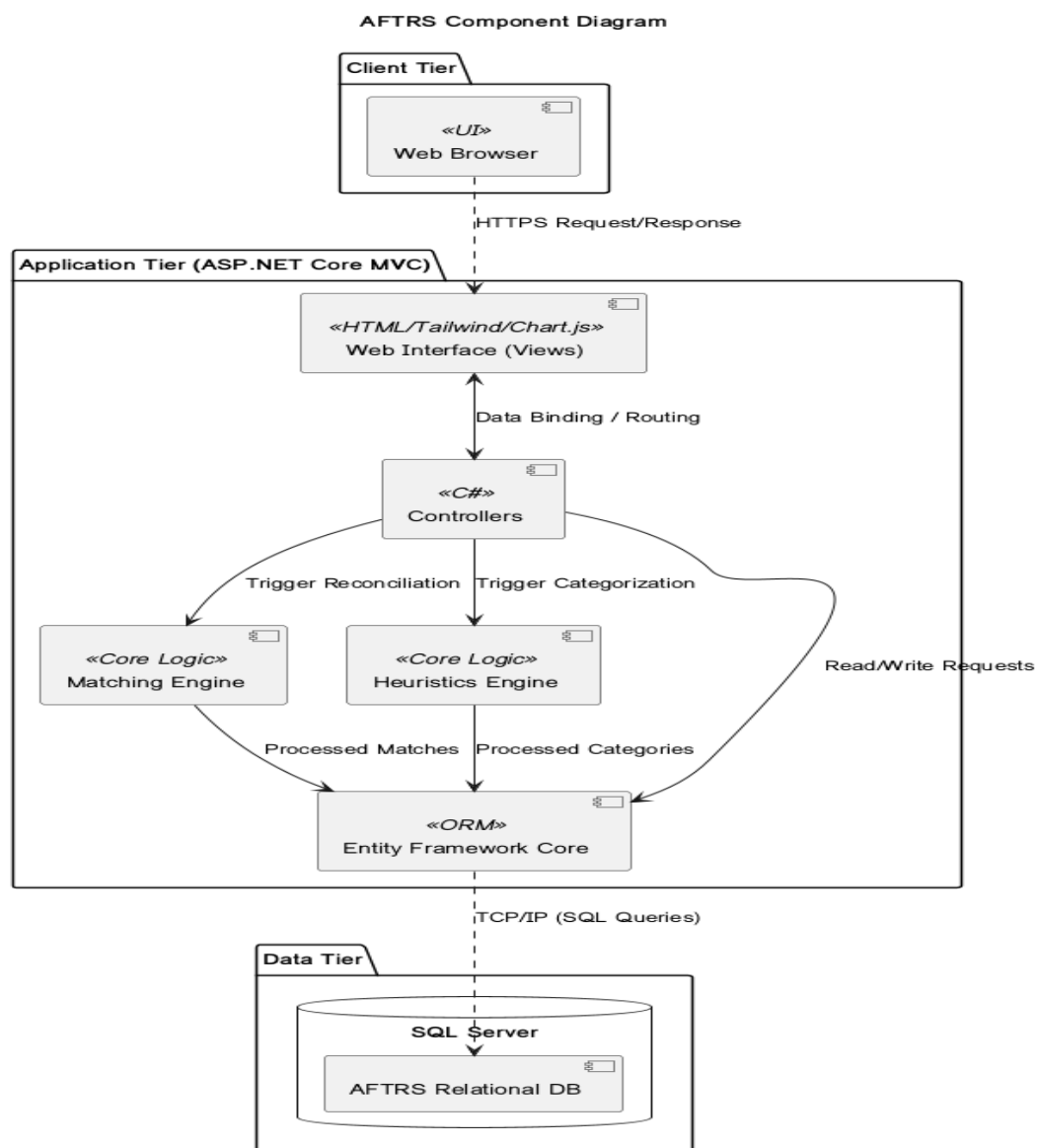


8. Component diagram

The Component Diagram provides a high-level, modular view of the AFTRS architecture. It breaks the complex application down into interchangeable, logical components. By adopting the ASP.NET Core MVC framework, the system is highly cohesive but loosely coupled.

The diagram visualizes three primary tiers:

- **Client Tier:** The user's web browser rendering the HTML/CSS Views.
- **Application Tier:** The C# Controllers, the Entity Framework Core (ORM) data access layer, and the core algorithmic engines (Matching Engine and Heuristics Engine).
- **Data Tier:** The Microsoft SQL Server housing the relational data and immutable audit logs.

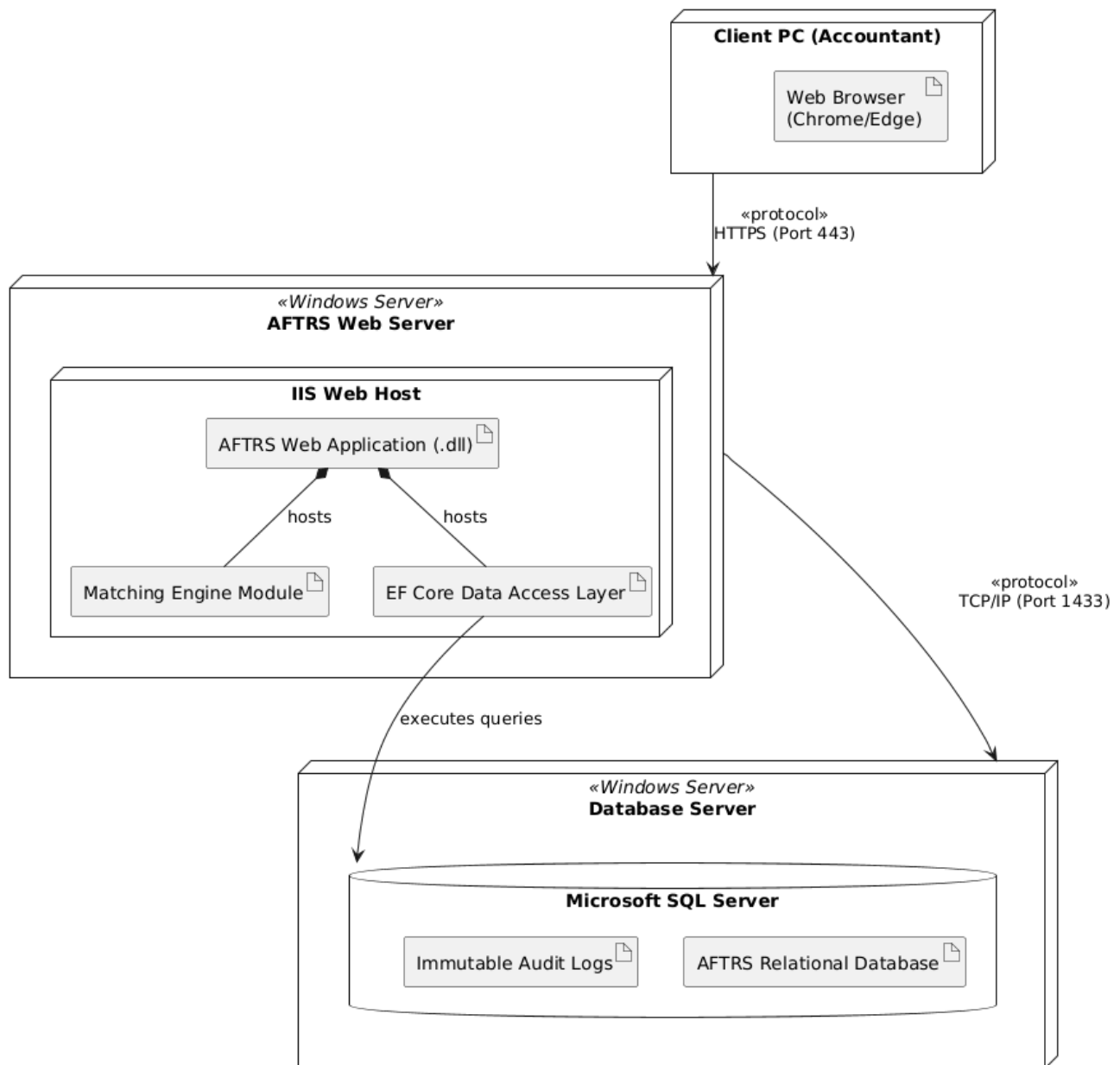


9. Deployment Diagram

The Deployment Diagram maps the software architecture onto the physical hardware infrastructure. It shows the topological layout of the physical nodes (servers and client PCs) and the communication protocols connecting them.

The AFTRS is designed for a secure, localized intranet deployment. The diagram illustrates the Client PC communicating via secure HTTPS (Port 443) to the Windows Server hosting the application via Internet Information Services (IIS). The web server then communicates with the dedicated Database Server via TCP/IP (Port 1433), ensuring a secure, multi-tier deployment that protects the financial data.

AFTRS Deployment Diagram



10. Reference

- [1] Object Management Group (OMG), "Unified Modeling Language (UML) Specification," Version 2.5.1, [Online]. Available: <https://www.omg.org/spec/UML/>
- [2] M. Fowler, UML Distilled: A Brief Guide to the Standard Object Modeling Language, 3rd ed. Boston, MA: Addison-Wesley Professional, 2003.
- [3] Microsoft Corporation, "Overview of ASP.NET Core MVC," Microsoft Learn, 2025. [Online]. Available: <https://learn.microsoft.com/en-us/aspnet/core/mvc/overview>
- [4] Microsoft Corporation, "Entity Framework Core Architecture and Modeling," Microsoft Learn, 2025. [Online]. Available: <https://learn.microsoft.com/en-us/ef/core/>
- [5] R. Elmasri and S. B. Navathe, Fundamentals of Database Systems, 7th ed. Hoboken, NJ: Pearson, 2015.
- [6] I. Sommerville, Software Engineering, 10th ed. Boston, MA: Pearson, 2015. (Note: Referenced for standard software architecture and component design principles).